Gradient Boosting

2026-04-17

Introduction

Gradient boosting, formalised by Friedman in 2001, builds an additive ensemble $F_M(x) = \sum_{m=1}^{M} \nu h_m(x)$ by fitting each weak learner h_m to the negative gradient of the loss at the current predictions. Decision trees are the canonical weak learner, the learning rate ν shrinks each tree's contribution to control regularisation, and the resulting framework dominates structured tabular-data benchmarks. Modern implementations (XGBoost, LightGBM, CatBoost) add second-order information, regularisation, missing-value handling, and other improvements over the original Friedman gradient boosting.

Prerequisites

A working understanding of decision trees, gradient descent, and the bias-variance trade-off in ensemble methods.

Theory

The generic gradient-boosting algorithm:

1. Initialise $F_0(x) = \arg\min_\gamma \sum_i L(y_i, \gamma)$ — the constant prediction minimising the loss.
2. For $m = 1, \dots, M$:
   - Compute pseudo-residuals $r_{im} = -\partial L(y_i, F(x_i))/\partial F(x_i)|_{F=F_{m-1}}$.
   - Fit a weak learner h_m to the pseudo-residuals.
   - Update $F_m(x) = F_{m-1}(x) + \nu h_m(x)$.

For squared-error loss the pseudo-residuals are ordinary residuals; for binary log-loss they are score residuals; for Huber loss they combine the two regimes. The learning rate ν trades fit quality for over-fit risk: smaller ν requires more trees but generalises better.

Assumptions

The chosen weak learner is appropriate for the loss (shallow trees with depth 3–6 for most losses); the learning rate is small enough for stable optimisation; sufficient sample size for the boosting interactions to be supported by the data.

R Implementation

```
library(gbm)

set.seed(2026)
n <- 500
df <- data.frame(
  x1 = rnorm(n), x2 = rnorm(n), x3 = rnorm(n),
  y  = sin(rnorm(n)) + 0.5 * rnorm(n)^2
)

m <- gbm(y ~ ., data = df,
```

```
          distribution = "gaussian",
          n.trees = 2000,
          interaction.depth = 3,
          shrinkage = 0.01,
          cv.folds = 5,
          verbose = FALSE)

best_iter <- gbm.perf(m, method = "cv", plot.it = FALSE)
cat("Best iter:", best_iter, "\n")
summary(m, n.trees = best_iter, plotit = FALSE)
```

Output & Results

`gbm()` with `cv.folds` returns the cross-validated boosting iteration count along with the variable-importance summary. `gbm.perf()` finds the iteration that minimises CV error; predictions at that iteration count are typically used for deployment.

Interpretation

A reporting sentence: "Gradient boosting with `shrinkage = 0.01`, `interaction.depth = 3`, and 5-fold cross-validated early stopping converged at 1{,}240 trees (CV RMSE 0.34); variable-importance scores attributed 52 % of relative influence to x_1, 28 % to x_2, and 20 % to x_3, consistent with the simulated data structure." Always state the shrinkage, depth, CV-optimal iteration count, and importance scores.

Practical Tips

- Smaller learning rate (0.01–0.05) with many trees almost always generalises better than larger rate with few trees; this is the canonical regularisation trade-off in boosting.
- Use shallow trees (depth 3–6) as the standard weak learners; deeper trees approach a single-tree model and lose the boosting benefit.
- `cv.folds = 5` in `gbm()` produces an early-stopping estimate without separate plumbing; `gbm.perf(method = "cv")` finds the optimal iteration count.
- For structured prediction problems, prefer XGBoost, LightGBM, or CatBoost over base `gbm`; they add regularisation, missing-value handling, GPU support, and substantially faster training.
- Monotone constraints (supported by XGBoost and LightGBM) embed domain knowledge into boosting; useful in regulated industries (credit, insurance, healthcare).
- Pair gradient boosting with feature-importance and partial-dependence plots for interpretation; raw boosted trees are otherwise opaque.

R Packages Used

`gbm` for the original Friedman gradient boosting; `xgboost`, `lightgbm`, `catboost` for modern alternatives with regularisation and speed; `parsnip::boost_tree()` for tidymodels integration; `mboost` for component-wise gradient boosting on additive models.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis